Pandas 2.0 and its Ecosystem (Arrow, Polars, DuckDB)

Simon Späti • March 6, 2023 • *7 min*

Data manipulation and analysis can be challenging and involve working with large datasets. Thankfully, a widely used Python library known as Pandas has become the go-to tool for processing and manipulating data. Pandas recently got an update, which is version 2.0. This article takes a closer look at what Pandas is, its success, and what the new version brings, including its ecosystem around Arrow, Polars, and DuckDB.

Pandas has established itself as the standard tool for in-memory data processing in Python, and it offers an extensive range of data manipulation capabilities. As such, it is unsurprising that data engineers or those just starting with data have come across it at some point in their work.

In the recent update, Pandas 2.0 has adopted Apache Arrow as a backend. This article explores why this is the most significant change in the update and how Arrow has become a fundamental part of many recently released frameworks and hot topics in data engineering.

That makes it an excellent time to reflect on what Pandas is and why it's successful.

> 🍿 Interesting to see the ecosystem evolving around Apache Arrow
> Many recently released frameworks connect to Arrow in one way or another. To name a few: Polars, DataFusion, Blaze, Cube's Cache Layer Cube Store, Dremio. But why? Let's discuss that.

What is Pandas

But let's start with what Pandas is? Pandas is an library to **process data in memory** with a feature-rich Python API. Compared to many tools, Pandas is *the* python library to work with smaller data and

sets the standard for wrangling data in memory.

Pandas is generally suited for:

- Tabular data, think of Excel as an SQL table
- Fast way to observe or do statistical analysis on smaller data sets
- Time series data ordered and unordered
- Arbitrary matrix data with row and column labels
- Local data pipelines to do transformation in-memory

Some key features you might have used it for:

- Easy handling of missing data
- Size mutability: columns can be inserted and deleted from DataFrame and higher dimensional objects
- Flexible group by functionality and handy slicing, fancy indexing, and subsetting features
- Merging and joining data sets
- Extensive and robust IO tools for loading data from flat files, Excel files, databases, etc.
- Lots of time series functionality

How Does Pandas Work?

To understand what is new and better with the latest version, let's briefly discuss how Pandas works.

Before doing anything with Pandas, the general idea is to load data in-memory **into a Pandas DataFrame**, usually using functions like read_csv, read_sql, read_parquet, etc. When loading data into memory, it must decide how it will be stored in memory.

For simple data like integers of floats, this is standard and straightforward. But some decisions must be made for other types, such as strings, dates and times, and categories.

Python is versatile and can represent mostly anything, but Python data structures (lists, dictionaries, tuples, etc.) are very slow and can't be used. So the data representation is not Python and is **not standard**, and implementation needs to happen via Python extensions, usually implemented in C (also in C++, Rust, and others).

For many years, the main extension to represent arrays and perform operations on them quickly has been NumPy. Which is also what Pandas was initially built on.

While NumPy has been good enough to make pandas the popular library it is, according to Marc Garcia (pandas core developer), it was never built as a backend for DataFrame libraries, and it has some critical limitations.

What Are the Highlights of Version 2.0

That brings us to version 2.0. Let's look at the key improvements: what are the highlights, and why do we have them?

- Representation of "Missing values" (None) and use of better support for data types outside of numerical types
- Faster speed
- Better interoperability

All of these are achieved through using Apache Arrow as the backend. By moving from NumPy (C++) to Apache Arrow as a backend (especially In Pandas 1.5 and 2.0 added Arrow support for all data types), Arrow gives it better storage and speed. A significant milestone was implementing a string data type based on Arrow that started in 2020.

Before we dive into a code example and why Arrow helped across the board, here is a short comparison of how the speed improved:

Operation	Time with NumPy	Time with Arrow	Speed up
read parquet (50Mb)	141 ms	87 ms	1.6x
mean (int64)	2.03 ms	1.11 ms	1.8x
mean (float64)	3.56 ms	1.73 ms	2.1x
endswith (string)	471 ms	14.9 ms	31.6x

Speed comparison between NumPy and Arrow by Marc Garcia.

What Changes Code-Wise?

The change of the Arrow type results in an extension of the type pyarrow. The pyarrow-addition was chosen to avoid breaking changes to existing code.

```
pandas.Series([1, 2, 3, 4], dtype='int64[pyarrow]')
pandas.Series(['foo', 'bar', 'foobar'], dtype='string[pyarrow]')
```

Arrow also defines a type to encode categories:

```
articles = pandas.DataFrame({
    'title': pandas.Series(['pandas 2.0 and the Arrow revolution',
                            'What I did this weekend'],
                           dtype='string[pyarrow]'),
    'tags': pandas.Series([['pandas', 'arrow', 'data'],
                           ['scuba-diving', 'rock-climbing']],
                          dtype=pandas.ArrowDtype(pyarrow.list_(pyarrow.string()))),
    'date': pandas.Series([datetime.date(2023, 2, 22),
                           datetime.date(2022, 11, 3)],
                          dtype='date32[pyarrow]')
})
```

What is Apache Arrow?

Apache Arrow sets the open standard to exchange in a heterogeneous data pipeline, which needs to read and share data among different steps.

Instead of the default, to persist copies along the way and lose costly time and resources to write and read that data, Apache Arrow allows sharing data in an in-memory representation without persisting.

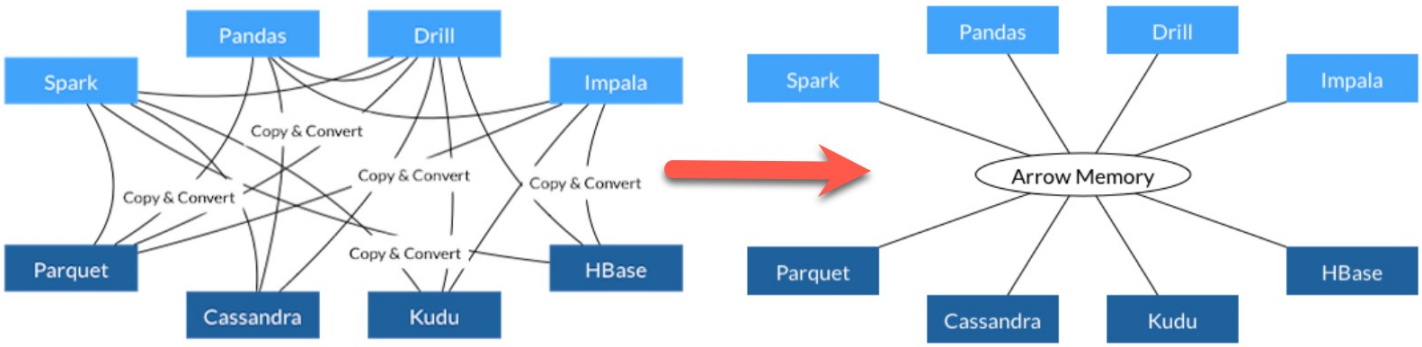
Why Apache Arrow?

Apache Arrow solves most discussed problems, such as improving speed, interoperability, and data types, especially for strings. For example, the new string[pyarrow] column type is around 3.5 times more efficient.

These efficiency gains also greatly influence how much data you can load with the same amount of RAM, which is essential when most Pandas DataFrames run on single laptops.

One of the biggest problems of Pandas was data interchange, particularly moving large tabular datasets from one process's memory space to another's.

The significant achievement here is **zero-copy data access**, mapping complex tables to memory to make accessing one terabyte of data on disk as fast and easy as one megabyte.



Arrow ecosystem and standardization from Overview Apache Arrow.

Another big one is above standardization saves. Apache Arrow has a vast ecosystem, and you can share it among all other libraries that integrate with Arrow. Moreover, they can implement custom connectors for new libraries and systems such as Polars. On top of these savings, a standardized memory format facilitates the reuse of libraries of algorithms, even across languages.

Interoperability

Arrow is a program-independent format. Similar to Data Lake File Formats but less obvious, as it does not have a file extension attached and only exists in memory.

Its interoperability makes it relatively easy to share the data among different programs and is speedy and memory-efficient because two programs can share the same data, in this case, the same memory, without copying each program. Which is the dream of every data engineer, right?

When you have a data pipeline that loads some data from your data lake, transforms it, computes some analytics, and then exports and automatically creates a Data Asset that can be used for analytics, you usually have different tools, abstractions, or even teams in more prominent companies. If you do not need to persist the data set at each step, that's a tremendous boost in performance and cost, as you do not need to store so much data immediately.

> 💪 This becomes even more powerful as Arrow becomes *the* open standard for in-memory data representation.

When Not to Use Pandas

So when is Pandas not the right choice?

People who are used to SQL usually prefer something other than the API of Pandas. Too bloat and too many "gotchas" is what people say.

The main problem is that you know how to do it in SQL, but you need to fiddle that SQL into a strange syntax you must look up most of the time.

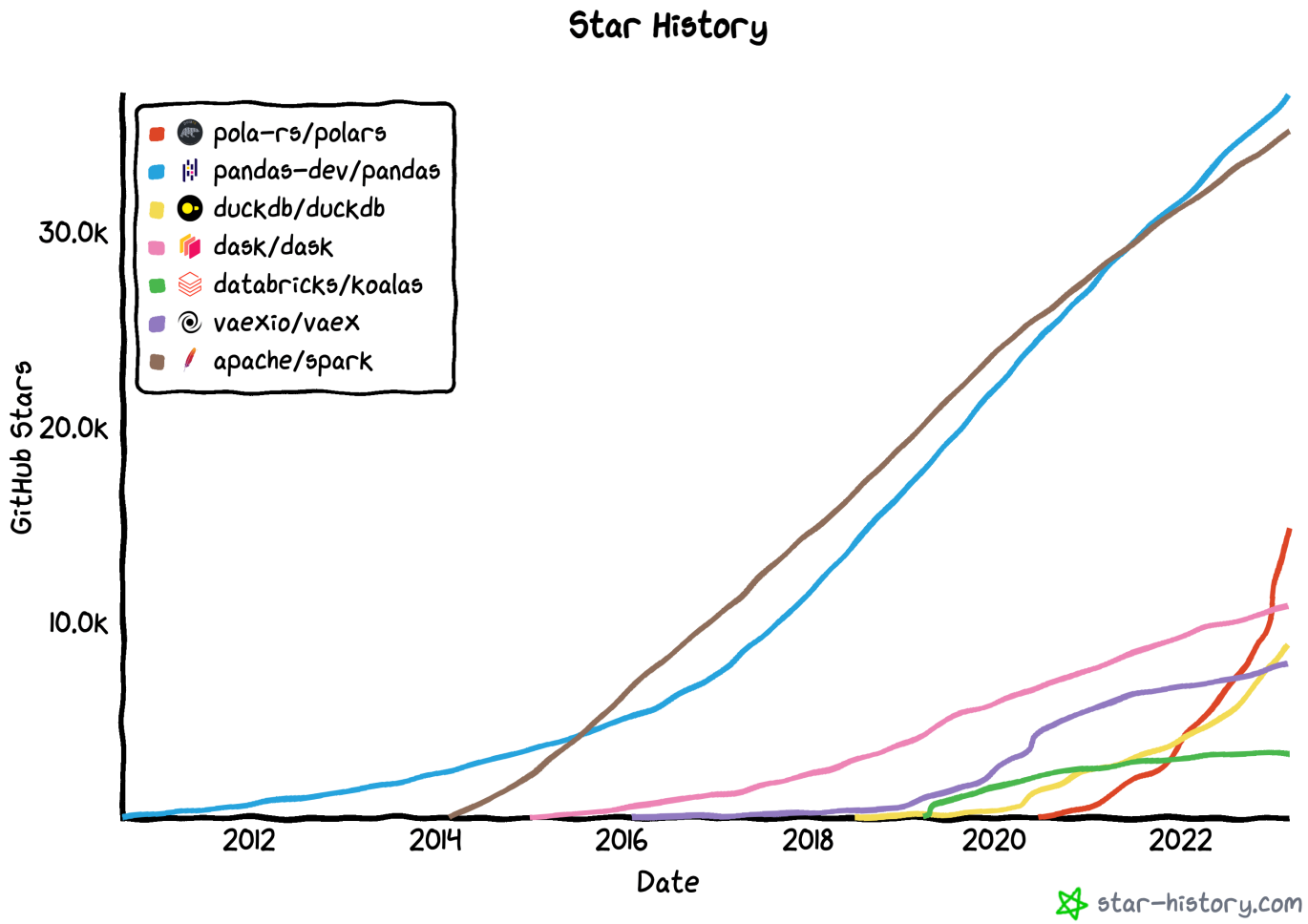
📝 11 Things Wes McKinney hated about Pandas (back in 2017)

1. Internals too far from "the metal"
2. No support for memory-mapped datasets
3. Poor performance in database and file ingest / export
4. Warty missing data support
5. Lack of transparency into memory use, RAM management
6. Weak support for categorical data
7. Complex group by operations awkward and slow
8. Appending data to a DataFrame tedious and very costly
9. Limited, non-extensible type metadata
10. Eager evaluation model, no query planning
11. "Slow", limited multicore algorithms for large datasets

But with the latest 2.0 release, many of these issues are handled, and with the integration of Arrow, future-proof and the cost of switching to another tool for a dedicated process is no problem anymore.

The Alternatives

The data ecosystem is growing every day, giving us plenty of alternatives to choose from. Let's look at these alternatives and try to understand why they were created and when they should be used.



Star history of GitHub repositories (Link or with aligned timeline)

Polars: Riding the Fast Train of Rust

The one-to-one replacement that provides a rich Python API would be Polars, implemented in Rust.

Some say Polars has a less confusing API and better ergonomics, especially from SQL. Polars is more performant out of the box but less stable and mature. It is growing the fastest of all mentioned in this chapter.

Polars has superpowers as it comes with a query optimizer that can make the pipeline run faster by analyzing all operations together before executing them.

> 📝 Good to Know
> When **sharing data between Pandas and Polars**, what Pandas 2.0 is doing is converting a PyArrow object into an Arrow2 object (or the other way round). There is little to convert since, internally, both libraries implement the exact data representation specification, the Apache Arrow specification. The bulk of the data, GBs or TBs, must not be serialized into another format. This allows data sharing easily, even with "big data."

An example of how that could look "switching" from one to another format:

```
loaded_pandas_data = pandas.read_sas(fname)

polars_data = polars.from_pandas(loaded_pandas_data)
# perform operations with pandas polars

to_export_pandas_data = polars.to_pandas(use_pyarrow_extension_array=True)
to_export_pandas_data.to_latex()
```

Daniel Beach says that Polars is the more accessible version of Spark and easier to understand than Pandas.

> 📝 Polars and Arrow
> Polars internal data representation is Apache Arrow, and now with Pandas 2.0, it's also one of the possible internal representations for Pandas DataFrames. To be entirely correct, it's Arrow2, a Rust structure to specific (there is another Arrow implementation in Rust, the official Arrow)

DuckDB: The SQL Version

DuckDB would be for SQL lovers. Sure, it's a database format, but in case you didn't know, DuckDB is also super strong as a zero-copy layer approach. For example, you can use DuckDB as a thin SQL wrapper on top of your S3 files across your Data Lake. DuckDB will do a great job of providing you with fast analytical queries.

DuckDB can efficiently run SQL queries directly on Pandas DataFrames. If you want to use SQL and have a fast interface, use DuckDB.

DuckDB can also query Arrow datasets directly and stream query results back to Arrow. This streaming allows users to query Arrow data using DuckDB's SQL Interface and API while taking advantage of DuckDB's parallel vectorized execution engine without requiring extra data copying. Additionally, this integration takes full advantage of Arrow's predicate and filter pushdown while scanning datasets.

More (how it works, benchmarks, example projects) about DuckDB on the Data Glossary.

What about Dask?

Many people may think of Dask as helping with Pandas performance and scalability. It optimizes speed by parallelizing large datasets into pieces and working with them in separate threads or processes or rescuing Pandas from the RAM limit.

One problem with the Dask is that it uses Pandas as a black box. dask.dataframe does not solve Pandas inherent performance and memory use issues. Still, it spreads them out across multiple processes. It helps mitigate them by being careful not to work simultaneously with too large pieces of data, which can result in an unpleasant MemoryError.

They make Dask jobs slower than possible with a more efficient in-memory runtime.

Others: Koalas, Vaex, VertiPaq

For completion, here are some pointers to other valid alternatives without explaining much more about them.

Koalas is another one, built on top of Spark, re-implemented 70%+ of the Pandas API by Databricks, and now officially included in PySpark since Apache Spark 3.2. Interesting to see that Spark has a similar growth rate of GitHub stars as Pandas, especially as Spark DataFrames are very similar to Pandas, supporting a rich Python API and SQL. But instead of stored locally, it's persisted inside the Spark cluster.

Vaex is a high-performance Python library for lazy **out-of-core DataFrames** (similar to Pandas) to visualize and explore big tabular datasets.

VertiPaq is the closed-source version from Microsoft. An engine is an in-memory columnar database behind Excel Power Pivot, SQL Server Analysis Services (SSAS) Tabular, and Power BI. When you

load data into a data model, it is loaded, compressed, and stored in RAM using the VertiPaq engine.

Data Lake Table Formats

Another way to solve this problem is using a Data Lake Table Format. Hence, you only read a fraction of the data to the pandas DataFrame, similar to what DuckDB is doing, but these formats are for large-scale and distributed files.

These have other advantages, such as predicate filtering and Z-ORDERING, which is more straightforward than adding a new tech like Polars, DuckDB, or others to the stack.

Conclusion

We have seen a brief on what Pandas is and how it works, highlighting its key features and capabilities. It has also touched on the limitations of its initial backend, NumPy, and how the move to Arrow in Pandas 2.0 addresses these limitations. Overall, this article provides insights into the benefits of using Pandas, particularly with its 2.0 version, and the exciting changes in its ecosystem around Arrow, Polars, and DuckDB.

Luckily whatever tool you use, once Pandas 2.0 is used, if you use Arrows types, converting between Pandas, Polars, and others will be almost immediate, with little metadata overhead.